Design Documentation

Interactive House

Revision History

Date	Version	Description	Author
01/03/2026	1.0	Initial design item descriptions focusing on requirements document	Katerina Arvay, Dario Ostojic, Andre Sandblom
17/03/2026	1.1	Added clarification of the current hybrid architecture and outlined planned future changes to the system design	Katerina Arvay, Dario Ostojic, Andre Sandblom

Design item List

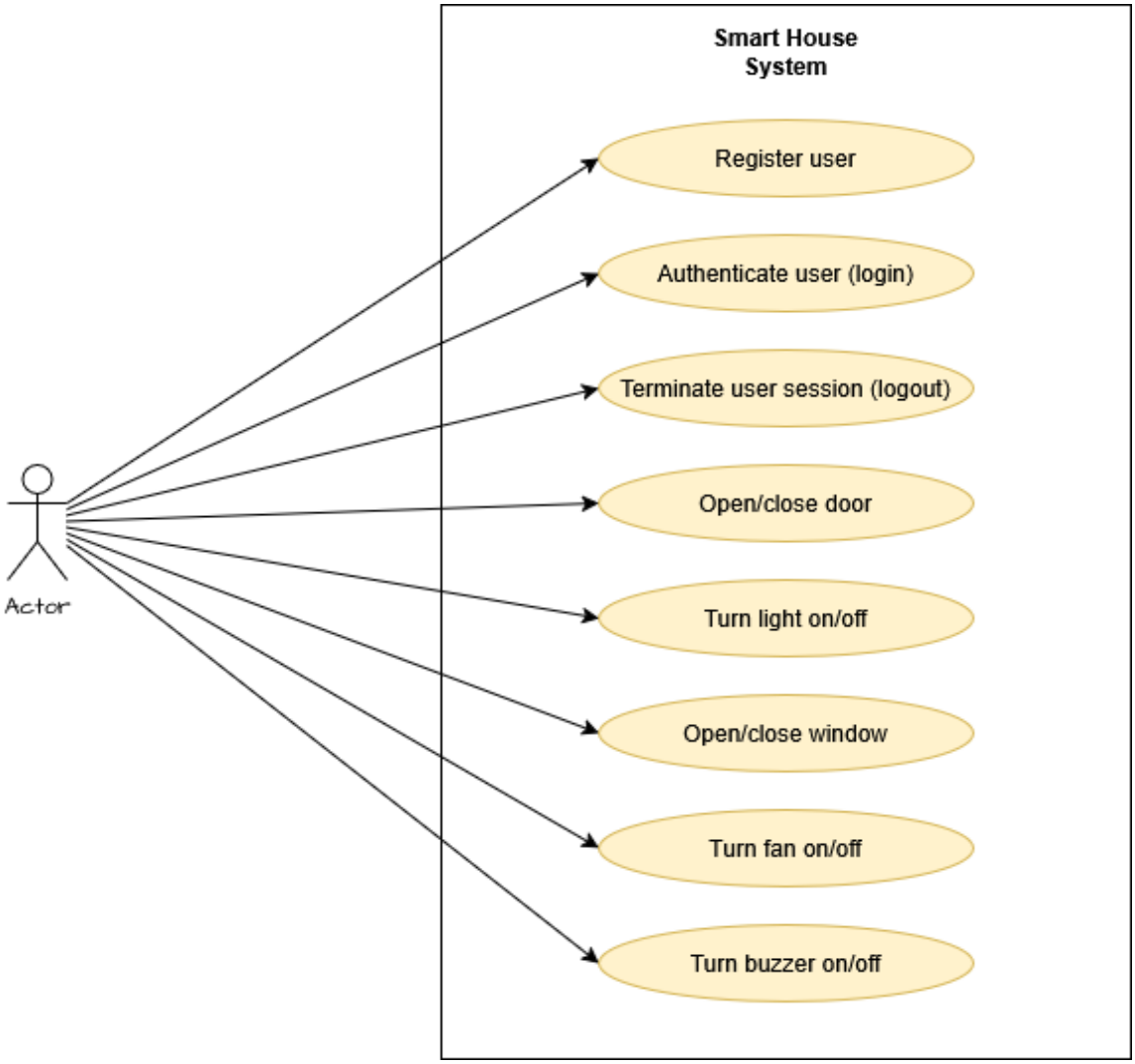
Requirement Name	Priority
D1. Use case diagram	Essential
D2. Entity and class diagram	Essential
D3. Interaction sequence diagram	Essential
D4. State diagram	Essential

Design Item Descriptions

D1. Use case diagram

The user can perform several key actions: registering a new account, logging in to authenticate themselves and logging out to terminate their session.

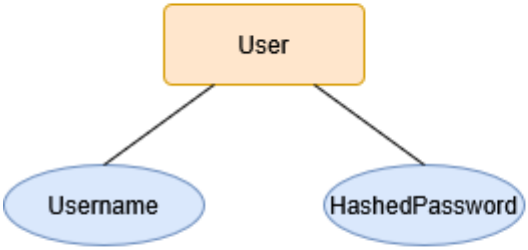
Once authenticated, the user can control various devices in the house. These include opening or closing the door and window, turning lights on or off, activating or deactivating the fan, and controlling the buzzer. Each of these actions corresponds to a specific use case from the system.

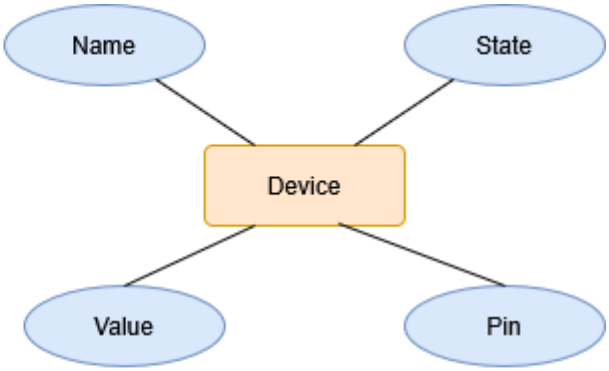


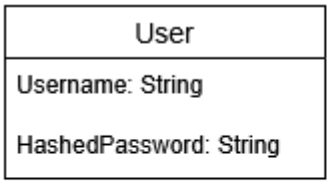
D2. Entity and class diagram

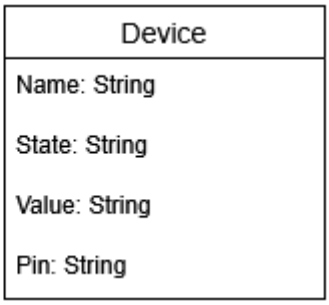
The User entity includes two attributes: Username, which uniquely identifies the user, and HashedPassword, which stores the user's encrypted password for secure authentication.

The Device entity represents smart home devices and contains four attributes: Name to identify the device, State to indicate its status (such as on/off or open/closed), Value to hold any additional relevant data and Pin which corresponds to the hardware control pin.









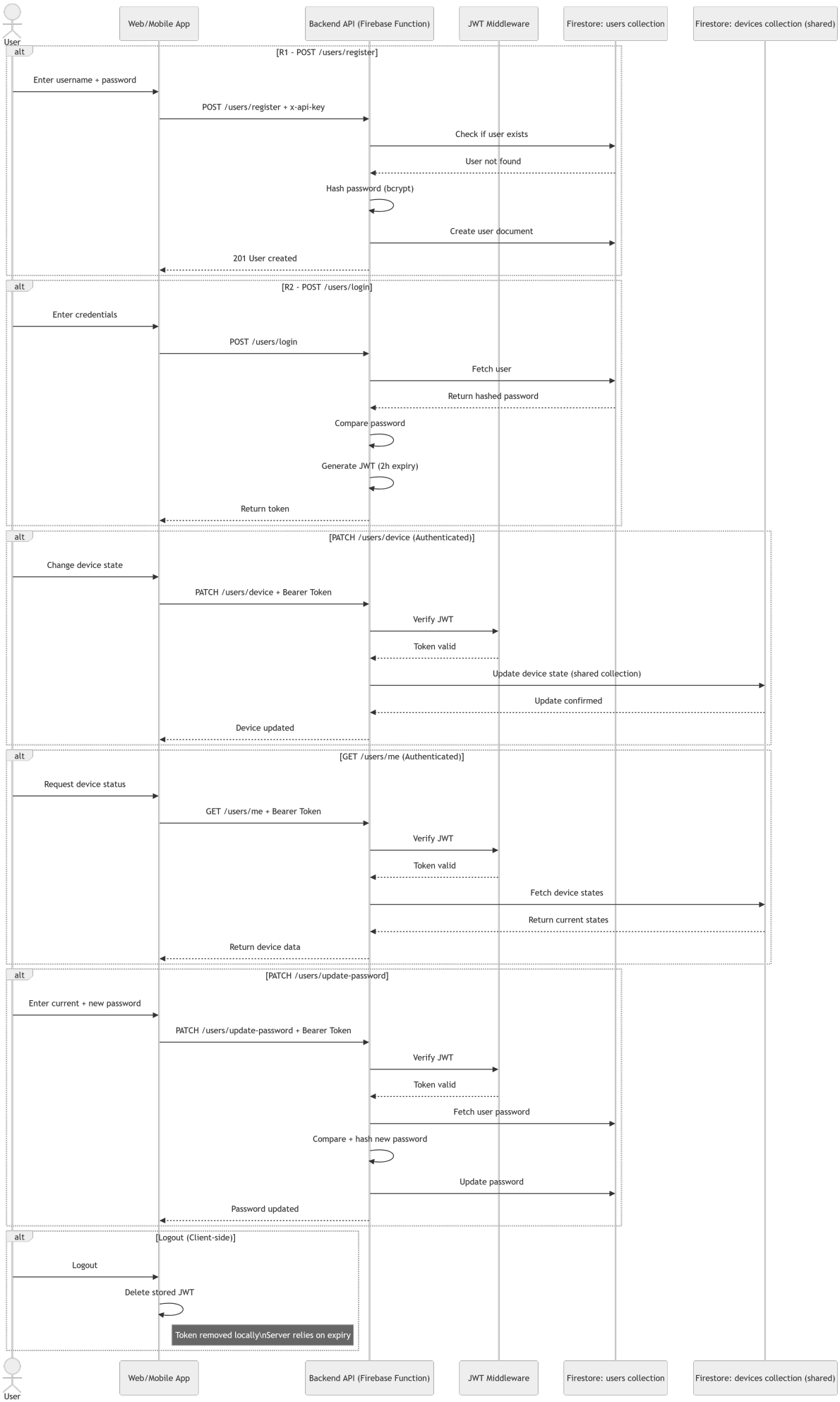
D3. Interaction sequence diagram

The sequence diagram shows how the user interacts with the Smart House system through a Web or Mobile application. When a new user registers, the application sends the username and password to the backend API. The server checks whether the user already exists, hashes the password and stores the user information in the users collection in Firestore. No devices are created during registration because all devices are stored in a separate, shared collection named devices.

When a user logs in, the application sends the credentials to the backend. The server verifies the password and, if it is correct, generates a JWT token. This token is required for all protected operations.

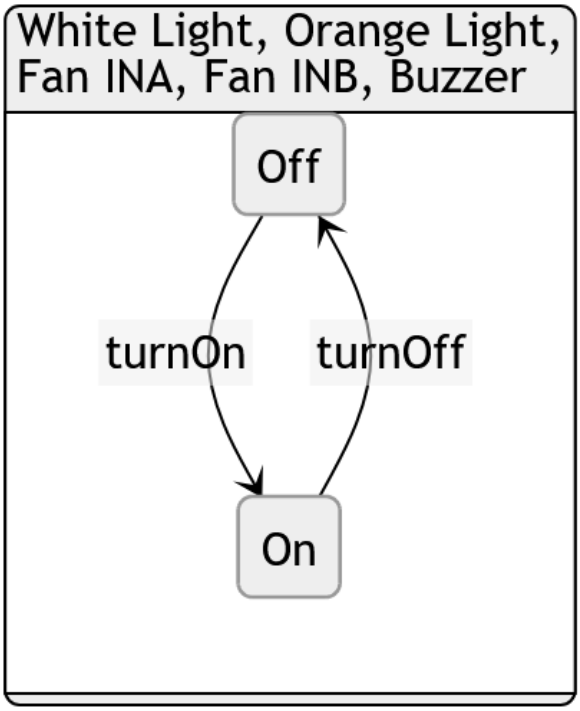
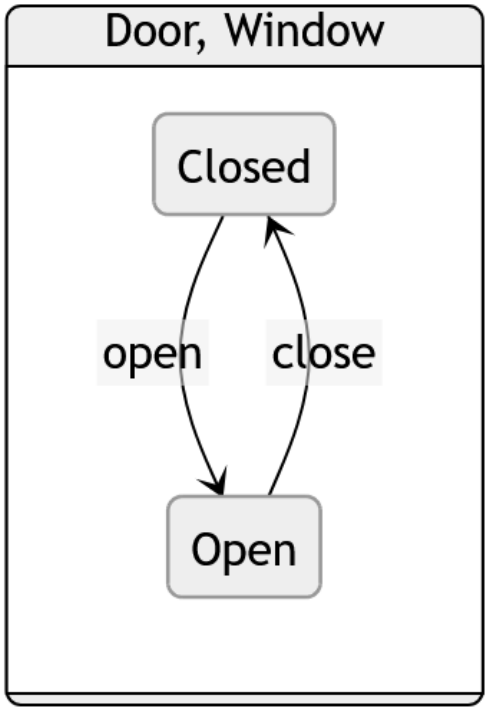
For actions such as turning a light on or off, opening or closing the door or window, activating the fan or buzzer, updating the password, or retrieving device status, the request must include the JWT token. The backend first validates the token and then performs the requested operation. Device states are updated or retrieved from the shared devices collection, while authentication data remains in the users collection.

Logout is handled on the client side by deleting the stored JWT token. After logout, the user must log in again to perform authenticated actions.


User
Web/Mobile App
Backend API (Firebase Function)
JWT Middleware
Firestore: users collection
Firestore: devices collection (shared)
alt
[R1 - POST /users/register]
Enter username + password
POST /users/register + x-api-key
Check if user exists
User not found
Hash password (bcrypt)
Create user document
201 User created
alt
[R2 - POST /users/login]
Enter credentials
POST /users/login
Fetch user
Return hashed password
Compare password
Generate JWT (2h expiry)
Return token
alt
[PATCH /users/device (Authenticated)]
Change device state
PATCH /users/device + Bearer Token
Verify JWT
Token valid
Update device state (shared collection)
Update confirmed
Device updated
alt
[GET /users/me (Authenticated)]
Request device status
GET /users/me + Bearer Token
Verify JWT
Token valid
Fetch device states
Return current states
Return device data
alt
[PATCH /users/update-password]
Enter current + new password
PATCH /users/update-password + Bearer Token
Verify JWT
Token valid
Fetch user password
Compare + hash new password
Update password
Password updated
alt
[Logout (Client-side)]
Logout
Delete stored JWT
Token removed locally\nServer relies on expiry
User
Web/Mobile App
Backend API (Firebase Function)
JWT Middleware
Firestore: users collection
Firestore: devices collection (shared)

In the current implementation, the system follows a hybrid approach where client applications communicate directly with Firebase Firestore, while a gateway component listens to database changes and synchronizes them with physical devices.

This differs slightly from the original design where a centralized backend API handled all interactions, and may be refined in future iterations.

D4. State diagram

Devices such as lights, fans and the buzzer follow the same On/Off behavior, while the door and window follow an Open/Closed behavior.